High-throughput Sampling, Communicating and Training for Reinforcement Learning Systems

Laiping Zhao†, Xinan Dai†, Zhixin Zhao†, Yusong Xin†, Yitao Hu*†, Jun Qian‡, Jun Yao‡ and Keqiu Li†
† College of Intelligence & Computing, Tianjin University, Tianjin Key Lab. of Advanced Networking, Tianjin, China
‡ Noahs Ark Lab, Huawei, Beijing, China
Email: {laiping, d1577707716, zhao612, avan, yitao, keqiu}@tju.edu.cn, {jack.qian, yaojun97}@huawei.com

2023 IEEE/ACM 31st International Symposium on Quality of Service (IWQoS) | 979-8-3503-9973-8/23/$31.00 ©2023 IEEE | DOI: 10.1109/IWQOS57198.2023.10188703

***Abstract*—Reinforcement Learning (RL) algorithms require large amounts of computational resources and time to train due to the simultaneous model training and real-time interactions with simulation environments. This results in an RL algorithm's training time potentially exceeding days to months.**

We present *HRL*, a comprehensive optimization system designed to improve the system throughput of RL algorithms. *HRL* addresses the challenges of discovering and identifying bottlenecks in the sampling, training, or communication stages and promptly improving their performance. We propose a group-parallel pipeline method to improve the sampling efficiency and apply data quantization and multi-learner training to resolve the network and learning bottlenecks. The *HRL* system has been fully implemented and integrated with *XingTian*. The results of the experiments show that the *HRL* system can improve the throughput by 18.6%-90.6%.

***Index Terms*—Throughput, Pipeline Sampling, Quantization, Reinforcement Learning System**

I. Introduction

Reinforcement learning (RL) algorithms have made great leaps in various areas, including Go chess [1], dialogue systems [2], autonomous driving [3], and robot manipulation [4]. However, such benefits come at a cost. Unlike typical deep learning models whose training data is prepared in advance, RL models are trained using trajectory data generated during the interactions with the simulation environment in real-time, which may take up to days or weeks to have sufficient samples for converging even on hundreds of CPUs and GPUs. For example, AlphaStar, a virtual character in the StarCraft2 game [5], relies on the A3C [6] model, which needs to be trained for 44 days to achieve superhuman capabilities using 3072 TPU cores and 50400 CPU cores. Likewise, in autonomous driving, if we use the PPO algorithm to train a high-quality driving policy that outputs the optimal steering commands in the driving scenario, it needs 2 Tesla K40 GPUs to train about 10 days [7].

Hence, it is crucial to enhance the system throughput (i.e., the number of samples processed per unit time) as well as the convergence speed of RL models. Prior work primarily focuses on improving the system throughput in two ways: (1) Innovations in the algorithm. For example, using asynchronous communication instead of synchronous communication. They can improve the throughput by reducing the waiting time between the training node and sampling node [8]–[10]. (2) Optimization of resource efficiency. That is, deploying more explorers, learners, or data transfer tasks on limited resources without degrading the performance. The resources could include the network [11], [12], CPUs and GPUs [13]–[15], etc.

Although prior work has improved the RL system throughput significantly, their application scenarios are very limited. For example, low-precision RL [12], [16], [17] can only improve the throughput when the network resources become a bottleneck; Sample Factory [14] and *GA3C* [13] take effect only when sampling and training become a bottleneck, respectively. Our experience in RL training shows that the bottleneck may occur in multiple places under different settings. For example, the bottleneck in the throughput of IMPALA tasks occurs at the *learner* when interacting with the CartPole environment [18], while it occurs at the *explorer* when interacting with the Atari environment [19]. Therefore, it is difficult to achieve high throughput only through a local optimization technique.

In this paper, we explore methods for comprehensively improving the system throughput considering the bottlenecks that may occur in various places, including learning agents, environmental interaction modules, or network communication. However, a high-throughput RL system introduces several challenges that need to be addressed. First, it needs the ability to promptly discover and identify bottlenecks that occur in the sampling, training, or communication stages; Second, for these bottlenecks, it should have effective optimization techniques to promptly improve the performance of sampling, training, or communication; Finally, optimizing any bottleneck will incur the cost, and the performance of each module also needs to be balanced to avoid additional bottlenecks caused by performance differences.

To overcome the above-mentioned challenges, we build *HRL*, the first comprehensive optimization system that can improve the throughput of RL systems by up to 90%. *In the context of the SpaceInvaders game [19], while achieving the same near-zero errors decision-making capability, the adoption of HRL can reduce the training time from 105 to 42 minutes using the asynchronous PPO algorithm [20]. HRL* has

*Corresponding author.

been fully implemented and integrated with *XingTian*[1] [21], *XingTian* is an RL framework created by *Huawei* that allows for the efficient training and deployment of RL models using multiple machine learning frameworks like *TensorFlow* [22], *MindSpore* [23].

and helps the development and verification of reinforcement learning algorithms.

Our contributions can be summarized as follows:

1) We provide a holistic analysis of the bottlenecks in the RL training procedure and find that bottlenecks may occur at any location in a system, and it is essential to consider all the potential sources of bottlenecks when optimizing the system performance.
2) We have proposed a group-paralleled pipeline method to improve the sampling efficiency and further apply the data quantization and multi-learner training into the system for resolving the network and learning bottleneck, respectively.
3) We design a coordination method to make the multiple running modules in RL system work together to maximize the throughput with minimal cost.
4) We completely implement *HRL* based on Xingtian and demonstrate its significant improvement in system throughput. The experiment results show that it can improve the DQN throughput by $31.7\% - 32.8\%$ on average, PPO by $18.7 - 90.6\%$, and IMPALA by $18.6\% - 52.9\%$.

II. Background and Motivation

A. RL Architecture

RL uses the Markov Decision Process theory to solve decision problems [24]. Modern deep reinforcement learning combines deep networks with traditional RL, enhancing learning and decision-making capabilities. Unlike supervised ML, which trains with pre-collected data, RL trains and collects data during the training process. This dynamic process makes RL more challenging, as the behavior of the simulation environment can be unpredictable, leading to longer training times ranging from milliseconds to days.

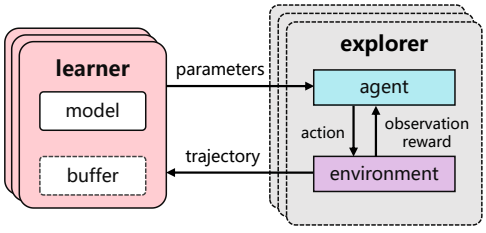


Fig. 1. The training procedure of RL

The basic architecture of the RL system consists of three parts: the *learner*, *explorer*, and *network*. Fig. 1 illustrates the typical architecture of the RL system. The *explorer* interacts with the environment and collects trajectory data. The *network* transmits the trajectory from the explorer to the learner and synchronizes the model between the learner and explorer. The *learner* trains the model using the collected trajectory data and outputs the final policy. The standard procedure of training in RL comprises four steps: *training*, *weight synchronization*, *sampling*, and *data transmission*. These steps are logically interdependent and occur in a specific sequence. Any bottleneck in any step will negatively impact the overall training performance.

B. Bottleneck Analysis

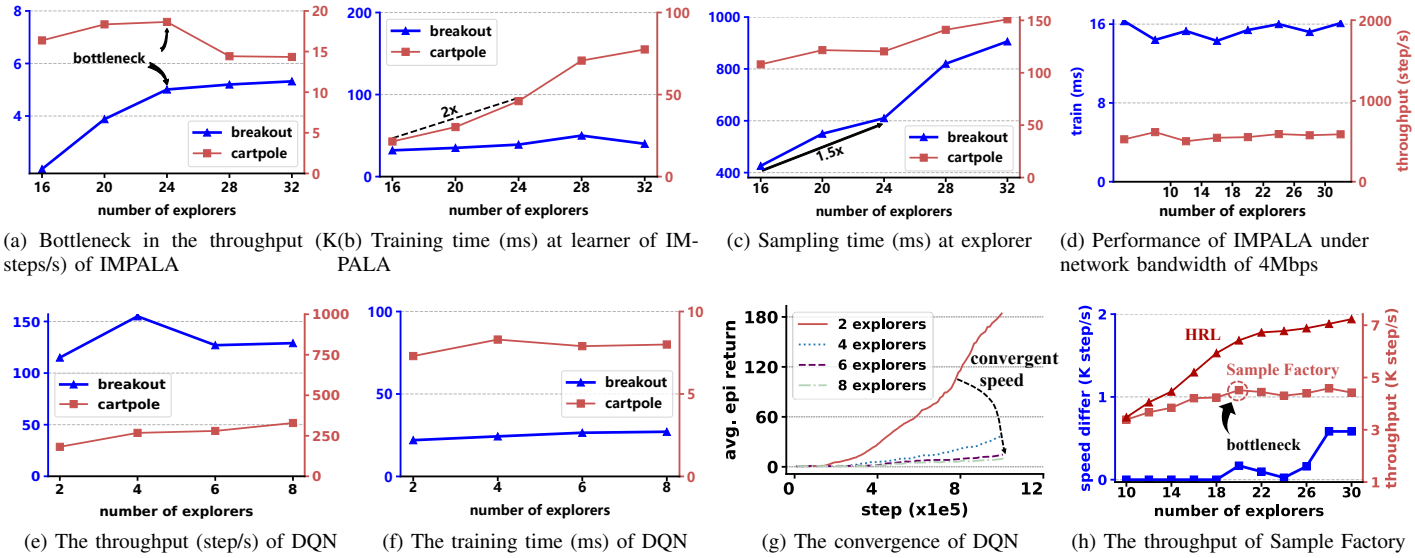
To maximize the system throughput and improve the training speed, it is crucial to analyze where bottlenecks occur, so that appropriate adjustments can be made to mitigate or eliminate these issues. In this section, we conduct a series of experimental analyses to identify bottlenecks in RL systems. We deploy two typical RL algorithms, IMPALA [8] and DQN [25], on the open-source RL framework *XingTian* [21]. Each algorithm are trained to generate movements in two simulation environments: the classic control environment CartPole in Gym [18] and the arcade learning environment Breakout in Atari [19]. The other software or hardware configurations refer to section V.

Observation 1: *The bottleneck in the throughput of the same model may vary over the simulation environments.*

We evaluate the IMPALA in both breakout and CartPole environments. IMPALA belongs to asynchronous algorithms, i.e., the training at the learner and the sampling at the explorer do not wait for each other. We increase the number of explorers from 16 to 32 to generate more training samples and identify the location of system bottlenecks. As the size of the buffer is limited, *XingTian* sets an upper bound on the number of explorers to make sure that all samples can be processed by learners. We remove the upper bound by increasing the batch size of the learner when the number of explorers exceeds the bound. For CartPole, we set the batch size of the learner to $\{1, 1, 2, 4, 6\} \times 160$ when the number of explorers increases from 16 to 32. For Breakout, as its data generation speed is far slower than that of CartPole, it is not necessary to increase the batch size.

Fig. 2a-2c shows how the overall throughput, training time, and sampling time change over the number of explorers. Ideally, more explorers would generate more samples, thus increasing the overall training throughput. However, this conclusion is not valid in practice. As shown in Fig. 2a, the peak is reached at 24 explorers. We analyze the training time at learner and the sampling time at explorer. Fig. 2b shows that, while the training time of Breakout constantly fluctuates between 30 and 40ms, the training time of CartPole increases significantly over the number of explorers due to the increased batch size. The rapidly increasing time offsets the improvement of throughput. Hence, the learner becomes a bottleneck when increasing the number of explorers in CartPole. For Breakout, its computation complexity is more complex than that of CartPole. As shown in Fig. 2c, when we deploy more explorers in the same server, the sampling time increases from 400ms to 900ms due to

[1] https://github.com/huawei-noah/xingtian



(a) Bottleneck in the throughput (steps/s) of IMPALA

(K(b) Training time (ms) at learner of IMPALA

(c) Sampling time (ms) at explorer

(d) Performance of IMPALA under network bandwidth of 4Mbps

(e) The throughput (step/s) of DQN

(f) The training time (ms) of DQN

(g) The convergence of DQN

(h) The throughput of Sample Factory

Fig. 2. Bottlenecks of the IMPALA model vary over the simulation environments. (a): The overall throughput does not always increase with the number of explorers; (b): For CartPole, the throughput no longer increases because the training time has increased much, i.e., the bottleneck occurs at the learner; (c): For breakout, the throughput no longer increases because the sampling time has increased much, i.e., the bottleneck occurs at the explorer; (d) The bottleneck is pushed to network under the network bandwidth limit of 4Mbps; (e) The throughput of DQN stays stable with the increase of explorers in both environments; (f) The train time of DQN stays stable with the increase of explorers in both environments; (g) The convergence speed decreases over the number of explorers due to the data; (h) The throughput improvement of Sample Factory is limited.

the interference among processes. Hence, in Breakout, the explorer becomes a bottleneck because the increasing sampling time offsets the improvement of throughput.

Observation 2: *Under the network bottleneck, the throughput cannot be improved through optimizing the explorers.*

We further evaluate the throughput of IMPALA by reducing the network bandwidth from 40Mbps to 4Mbps (i.e., creating a network bottleneck) in the Breakout environments. In this way, the updated model at the learner cannot be synchronized to explorers in time, and explorers have to wait for the updated model. Hence, the computation in the inference and environments at explorers becomes much less intensive. Fig. 2d shows that both the throughput and sampling time do not change much with the increasing number of explorers. That is, when the network bandwidth is insufficient, the bottleneck of IMPALA will be pushed from the explorer to the network communication.

Observation 3: *For value-based reinforcement learning algorithms, the system throughput and convergence speed may not improve with the number of explorers because the bottleneck occurs at the learner.*

DQN is a classic value-based reinforcement learning algorithm [25]. Unlike policy-based reinforcement learning algorithms, it relies on an experience replay buffer, which is used for storing the samples generated at explorers. During the training stage, learners only need to select data from the replay buffer. Hence, the sampling speed only affects the update speed of the experience replay buffer.

As shown in Fig. 2e and Fig. 2f, we find that both the throughput and the training time at learner do not change much when the number of explorers increases from 2 to 8, no matter in CartPole or in Breakout. The convergence speed even decreases over the number of explorers due to the data loss caused by the congestion in the replay buffer (Fig. 2g). Hence, for the value-based reinforcement learning algorithms that rely on a replay buffer, the bottleneck occurs at the learner.

Observation 4: *Due to the uncertain location of bottlenecks, a single optimization technique cannot guarantee an overall improvement in throughput.*

A number of techniques have been proposed to mitigate the bottleneck of RL systems [11], [12], [17]. We choose Sample Factory [14] and implement it in *XingTian*. It mainly improves the throughput in explorers through the multiple parallel environment workers and inference workers. Its inference workers are deployed on GPU, and the environment workers run on CPUs.

We evaluate its throughput under the different number of explorers (Fig. 2h). We find that the throughput does not increase after the number of explorers exceeds 20 because the samples saturate the learner (the sample generation rate exceeds the sample consumption rate after 20 explorers), making the learning become a bottleneck. If we further activate the optimization of communicating and learning using our *HRL*, we can see that the throughput is increased by 63%, compared with Sample Factory.

C. Summary

In RL systems, the location of bottlenecks is highly uncertain. It may occur in many places, such as the learner, the explorer or the network. Even for the same RL algorithm, the bottleneck location can change due to different configuration parameters (**Observation 1 and 2**). The bottleneck location can also vary for different types of algorithms (**Observation**

3). These features result in limited effectiveness of existing single optimization methods (**Observation 4**). Hence, we explore a holistic optimization method that can improve the overall throughput of RL systems.

III. System Design

In this section, we present the design of *HRL*. All notations are listed in Tab. I.

A. System Architecture

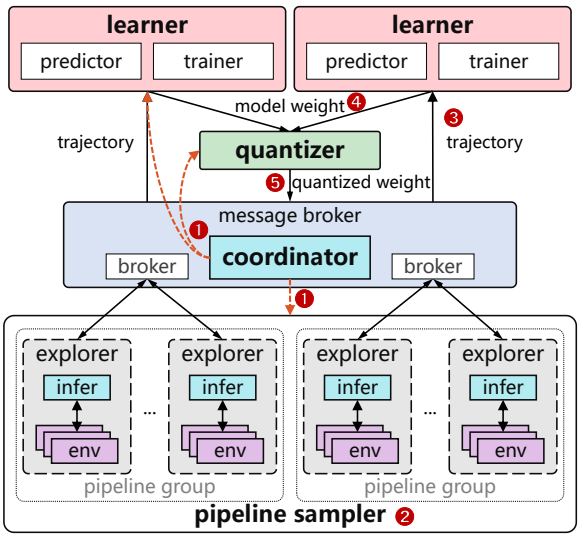


Fig. 3. The architecture of *HRL*.

The basic idea of *HRL* is that *the throughput can be improved through a combined optimization of sampling, communicating, and learning*. Fig. 3 highlights the design overview of *HRL*. In the explorer, we propose a *group-based pipeline sampling* method to improve the sampling efficiency. In the learner, we adopt a *multi-learner* design to improve the model training efficiency. For the data transmission between the learner and the explorer, we adopt the *data quantization* method to improve the network transmission efficiency. Finally, we design a *coordination* module to coordinate the three optimization methods for improving the overall throughput.

When a RL training task is started, the *coordinator* first derives the optimal global configuration based on the task configuration and hardware information and then starts the *pipeline sampler*, *quantizer*, and *learner* (❶). Next, the *explorers* in the *pipeline sampler* starts sampling in parallel to collect a certain number of trajectories (❷). These trajectories are gathered and distributed to multiple learners by the *message broker*, which is responsible for message serialization and transmission (❸). The *trainer* and *predictor* in the *learner* use the received trajectories to train and update the model and send the updated model weights to the *quantizer* for weight quantization (❹). The *quantizer* then sends the compressed model weights to the agent in each *explorer* via *message broker* for the next round of sampling and training (❺). During the pre-training phase, the *coordinator* will continuously monitor the system throughput and resource utilization and thus make fine-grained tuning to the *pipeline sampler* and *learner* configuration until one component of the system reaches its bottleneck.

B. High-throughput Pipeline Sampling

Sampling is an essential task in the explorer. It relies on the interactions between *inference* and *environment*. In particular, the *inference* makes decisions, and the *environment* actually executes the decisions. To improve the sampling throughput, it is common to deploy a large number of explorers. However, the existing RL systems adopt a *"one-to-one mapping"* method between *inference* and *environment*, i.e., each agent activates a separate *inference* module and interacts with a separate *environment* module. Due to the mutual waiting between each pair of *inference* and *environment*, the resource efficiency is actually very low in the explorer.

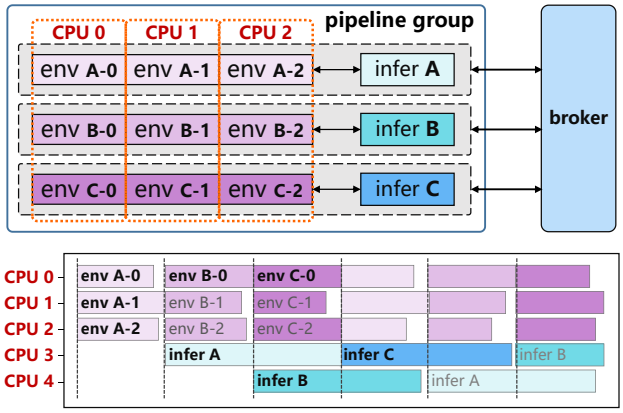


Fig. 4. The group-based pipeline sampling process.

We propose a *group-based pipeline sampling* method for improving the sampling throughput. As shown in Fig. 4, it divides the *inferences* and *environments* into m groups. In each group, there exists an *inference* and n *environments*, for enabling the batch processing of each *inference*. The m groups are deployed on a number of CPU cores and operate in a pipeline format for improving CPU efficiency. Denote by $E_A = \{A_0, A_1, A_2\}$, $E_B = \{B_0, B_1, B_2\}$ and $E_C = \{C_0, C_1, C_2\}$ the three groups in Fig. 4. Then, the execution process of a single round of pipeline can be described as follows: First, the environment processes in E_A are executed following the decisions of inference in the previous round. After completion, the data is handed over to the inference process in E_A for processing. At this time, the CPUs occupied by the environment processes in E_A are released, and the environment processes in E_B start to execute. This process continues in a loop until the environment process in E_C completes execution. At the same time, the inference for E_A completes and all environment processes in E_A start their executions again.

Denote by s_{pip} the sampling throughput of a pipeline. Then, we have,

$$s_{pip} = \frac{n}{t^n_{env}} \tag{1}$$

where t^n_{env} represents the processing time of a group of *environments* under n parallelism. As group size increases,

TABLE I
NOTATIONS

symbols	description
s	the overall throughput of the RL system
s_e	the sampling throughput at explorers
s_n	the network throughput
s_l	the training throughput at learners;
m	the number of groups in a pipeline;
n	the number of environments in a group;
s_{pip}	the sampling throughput of a pipeline;
t^n_{env}	the processing time of a group of n environments;
u_{avg}	the average throughput on a CPU core;
t^n_{inf}	the inference time under n parallelism;
c_{cpu}	the number of CPU cores allocated to a pipeline;
q	the precision that is used by the quantizer;
d_q	the model size after qquantization;
e	the network bandwidth between learners and explorers;
t_{trn}	the total time for running a training step;
b	batch size used in learning;
t_f	the forward propagation time for processing a mini-batch;
t_b	the backward propagation time;
d	the size of the training model;
g	the number of GPUs;
w	the bandwidth between GPUs;

waiting time also increases. To improve sampling throughput, multiple groups can be run simultaneously. This allows for the efficient collection of data from several groups at once.

As both *inferences* and *environments* in a pipeline consume CPU cores, they should maximize the utilization of limited CPU resources for maximizing the overall sampling throughput. Denote by u_{avg} the average throughput on a CPU core. Then, our objective is,

$$\max_n\{u_{avg}\} = \max_n\{\frac{s_{pip}}{c_{cpu}}\} = \max_n\{\frac{n}{t^n_{env}} / \left(n + \left\lceil \frac{t^n_{inf}}{t^n_{env}} \right\rceil\right)\} \tag{2}$$

where t^n_{inf} represents the inference time under n parallelism (i.e., the batch size is n), c_{cpu} denotes the number of CPU cores allocated to a pipeline, and $\left\lceil \frac{t^n_{inf}}{t^n_{env}} \right\rceil$ is the number of cores allocated to *inference* processes in a pipeline. As both t^n_{inf} and t^n_{env} can be obtained through pre-profiling, the variable of n can be easily derived through the derivative calculation.

Given C CPU cores in a cluster of explorers, then the upper bound of sampling throughput at explorers (denoted by s_e) can be derived as below,

$$s_e = \max_n \{u_{avg}\} \times \left\lfloor \frac{C}{c_{cpu}} \right\rfloor \times c_{cpu} + r \tag{3}$$

where r represents the throughput that can be achieved by the rest of the CPUs that are not allocated to the pipelines.

C. High-throughput Communication

The *communication* module is responsible for transmitting data samples to learners or synchronizing the updated model parameters to explorers. While the learning model is complex, the network may become a bottleneck. We mitigate the network bottleneck using *buffer* and *quantization*.

1) Buffer: The samples generated at *explorers* need to be transported in a timely manner. Otherwise, it will block the pipeline and degrade the sampling efficiency. To resolve this problem, we set two buffer queues in the communication layer. They are used to store the updated model parameters and data samples, respectively. As shown in Fig. 3, after the pipeline sampler generates samples, it sends them directly to the broker. Then, the broker stores the data in its buffer queue. The pipeline sampler also pulls the model parameters from the other broker buffer and performs the next sampling task.

2) Quantization: Model quantization refers to reducing the precision of the parameters of a machine learning model from high precision (e.g. 32-bit floating point, $fp32$) to low precision (e.g. 8-bit integer, $int8$), without significantly affecting the model's accuracy [12]. This can result in faster computations, lower memory usage and lower network bandwidth requirements. Denote by q the precision used by the *quantizer*. Then, we have,

$$q \in \{fp32, fp16, int8\} \tag{4}$$

Denote by s_n the network throughput, then we have,

$$s_n = \frac{d_q}{e} \tag{5}$$

where d_q is the model size after $q-$quantization, e represents the network bandwidth between learners and explorers.

A simple way to implement quantization in RL systems is to add an interval module responsible for compressing the model. However, directly adding a new module will introduce extra overhead into the iteration and reduce the performance of the total system. In some cases, the time consumed by model quantization is unacceptable in asynchronous RL algorithms. For example, the quantization of IMPALA from 32-bit floating point to 8-bit integer takes more than 200 milliseconds. We use a parallel quantization way to speed up this compressing process. The quantization module consists of a series of parallel quantitation processes and two queues that store the model for quantization and the quantized model, respectively. When the learner generates the model, the model will be put into the input queue and further quantized by an idle quantization process. In the compressing process, if another model is received, the new model needs not to wait for the previous process complete, but directly activate another idle process if available. Finally, explorers read the quantized model from the output queue.

D. High-throughput Multi-learner Training

Training tasks in RL have the same features as supervised learning. The RL algorithm defines the way the RL target is updated, and in this process, the RL model performs the forward and backward propagation tasks. As the samples collected from explorers increase, the demand for learners' processing capacity also increases. To solve this problem, we use distributed training to speed up the training process.

The workload stream in the training tasks begins with the arrival of trajectory data collected from explorer agents. The

data is split into different learners for further processing. Once the preparation is complete, the forward and backward propagation of the training will be performed. Periodic weight synchronization will occur among the GPU nodes during the training process. When all training tasks are finished, the final weights will be sent to the explorers for the next iteration of sampling.

We adopt the all-reduce architecture [26] to distribute the training tasks among multiple GPUs. Although this leads to a number of synchronization operations among GPU nodes, it also enables the learner to handle large datasets.

Denote by t_{trn} the total time for running a training step on a single GPU in the all-reduce architecture. Suppose the batch size in training is b, then we have,

$$t_{trn} = bt_f + t_b + \frac{2d(g-1)}{gw} \tag{6}$$

where t_f represents the forward propagation time for processing a mini-batch, t_b is the backward propagation time, d denotes the size of the training model, g is the number of GPUs, and w represents the bandwidth between GPUs. According to the all-reduce communication architecture [26], the communication during the training process mainly consists of two parts: *parameter aggregation* and *parameter distribution*. The overall communication time during the model training is $2d(g-1)/gw$.

Thus, the training throughput (i.e., s_l) for learners is,

$$s_l = \frac{bg}{t_{trn}} = \frac{bg}{bt_f + t_b + 2d(g-1)/(gw)} \tag{7}$$

E. Overall Coordination

The overall throughput of the RL system (i.e., s) is limited by the bottleneck that may occur in *learner*, *explorer* or *network*. That is, we have,

$$\max s = \max \min\{s_l, s_e, s_n\} \tag{8}$$

Given Formulas (3), (5), (7), we need to derive the variables of $< n, q, b >$ for achieving the optimal throughput. Considering that the data transmitted in different directions is not the same, we discuss the coordination method separately.

1) "Explorer → Network → Learner": As the overall throughput is limited by the bottleneck, it is not necessary for other components to be configured at their maximum throughput. Hence, this requires coordinating the configurations of the components so that their throughput is consistent. Given the overall throughput s, we have,

- Pipeline sampling. We derive the variable n that is able to establish the formula of $\max_n\{u_{avg}\} = (s - r)/(\lfloor C/c_{cpu} \rfloor c_{cpu})$.
- Multi-learner. We derive the variable $b = (t_b + 2d(g - 1)/(gw))/(g/s - t_f)$.
- Network. From *explorer* to *learner*, the network only needs to transmit the trajectory data. Hence, it does not need to configure the *quantizer*. Suppose the size of the trajectory data is d_{tra}, then the network can complete the transmission in time of d_{tra}/e, where e is the network bandwidth.

2) "Learner → Network → Explorer": From *learner* to *explorer*, the network only needs to transmit the model parameters for updating the *inference* model. We do not perform separate optimization for *learner* and *explorer* as the size of the model is constant. However, the *quantizer* is configured at the low precision for improving the efficiency of network transmission, as long as the low precision does not affect the model's accuracy.

IV. System Implementation

HRL has been fully implemented and integrated into *XingTian* [21], with approximately 5,000 lines of python code. The majority of code (i.e., about 90%) is used to extend *XingTian* to support the new features and functionality. The rest is used for scheduling optimization and system optimization. We also developed about 1500 lines of python and shell scripts for system performance evaluation, functional verification, and data collection. Currently, *HRL* supports *TensorFlow* and *Keras* models.

Coordinator: We replace the *Controller* module in *XingTian*, use the *intel-cmt-cat* toolkit to monitor and test network bandwidth and memory utilization, use the Linux performance analysis tool *Perf* to monitor CPU and cache status, and use NVIDIA System Management Interface (*nvidia-smi*) to obtain and monitor GPU status.

Pipeline sampler: We implement the pipeline sampler as a module with adjustable multi-dimensional configurations. We implement resource allocation by setting the CPU core affinity of the threads in *create_explorer* function, using the Linux system resource management interface and the *psutil* library for pipeline scheduling.

Envpool: To satisfy efficient environment parallelism at the thread level, we use the open-sourced RL environment parallel library envpool [27]. We schedule the execution of the environments at the thread level to achieve an efficient pipeline.

Quantizer: Once the quantized model weights are received, the explorers use *TFLite* interfaces to restore the model and perform further inference tasks. The quantizer is composed of three main functions: *CompressWeight* quantizes the model; *FixGraph* resizes the batch size, and *TaskSchedule* monitors the quantization workers' state, and invokes or suspends idle workers accordingly.

Multi-learner: We implemented multi-learner with a distributed training framework *KungFu* [28] and integrated it with the original training module of *XingTian*. Trainers can communicate with each other through either inner-GPU communication with shared memory or inter-GPU communication using the NCCL framework.

V. Evaluation

A. Experimental Setup

1) Testbed: We deploy *HRL* on a testbed equipped with 32 logical cores (Intel Xeon Silver 4215 * 2), 128GB memory and 3 NVIDIA GeForce RTX 2080Ti GPUs. The operating system is Ubuntu 18.04.

2) RL Benchmarks: We evaluate the throughput and convergence speed for training DQN, PPO, and IMPALA on *HRL*.

- DQN (Deep Q-Network), is an RL algorithm that uses deep neural networks to approximate the action-value function [25].
- PPO (Proximal Policy Optimization), is an RL algorithm that aims to solve the problem of stability and reliability of policy gradient methods while maintaining the high sample efficiency of value-based methods [20].
- IMPALA (Importance Weighted Actor-Learner Architecture) is an RL algorithm that uses a distributed architecture to parallelize the learning process [8].

In addition, we use two sets of simulation environments:

- Gym Atari environment [19] is a set of Atari 2600 environments simulated through Stella and the Arcade Learning Environment.
- Gym classic control environment [18] is a suite of continuous control tasks involving the control of a simulated physics engine to move a robotic arm or a cart-pole system to achieve a specific goal.

3) Performance Metrics: We mainly consider two metrics in evaluations.

- Throughput: the trajectory steps consumed by the learner [8], [14], [20], [21], [29].
- Average episode return (or reward/return): refers to the average total reward received by an agent over a single episode of interacting with an environment [6], [8], [13], [20], [25]. In the context of our experimental methodology, we have gathered a dataset consisting of 10 million observations from the Atari environment and 1 million observations from the Gym classic control environment to demonstrate the efficacy of the utilized algorithms.

4) Competing Systems: We compare *HRL* with Sample Factory [14], RLlib [9] and the original *XingTian* [21] because they are state-of-the-art for improving the RL throughput.

- Sample Factory (*SF*): is a RL system specific optimized for the asynchronous PPO (APPO) [14]. It utilizes an asynchronous, GPU-based sampler that is highly efficient and augmented with off-policy correction techniques.
- *XingTian*: is a RL system that codesigns the management of communication and computation [21]. It organizes the computation in RL algorithms in a decentralized way and provides an asynchronous communication channel.
- RLlib: is a DRL framework based on Ray [29] that provides scalable software primitives for RL [9]. It enables a broad range of algorithms to be implemented with high performance, scalability, and substantial code reuse.

B. Overall Throughput

We evaluate the system throughput and the average episode return by gathering statistics until the RL algorithm has reached convergence following a pre-defined number of training steps. To this end, we record the performance metrics of up to 10 million training steps for the Atari environment and up to 1 million training steps for the Gym classic control environment.

***HRL* increases the throughput of RL system in different scenes with unchanged convergence.** Fig. 5 shows the throughput of *HRL*, RLlib and the original *XingTian*. Compared with *XingTian*, *HRL* achieves an average throughput improvement of $31.7\%-32.8\%$ on average in DQN, $18.7-90.6\%$ in PPO, $18.6\% - 52.9\%$ in IMPALA. The DQN algorithm achieves improved throughput through optimization during its training process. Using a *multi-learner* with the PPO algorithm can reduce the training time, and optimizing communication pathways can increase throughput. The IMPALA algorithm benefits from increased trajectory generation speed provided by the *pipeline sampler*, and the *quantization* technique can effectively reduce data transmission latency.

Compared to RLlib, *HRL* still exhibits even higher throughput improvement. The improvement achieves 33.7-83.3% in DQN, 35.9-56% in PPO and 90-163% in IMPALA.

***HRL* brings $31.7\% - 32.8\%$ increase in DQN's throughput by mitigating the learner bottleneck.** As DQN has its experience replay buffer for storing data, the sampling speed will only affect the update speed of the replay buffer. The training speed of the learning model decides the throughput. So the *coordinator* applies the multi-learner optimization in the training process of DQN. Fig. 5 shows the improvement brings $31.7\% - 32.8\%$ increase in throughput of Atari environments.

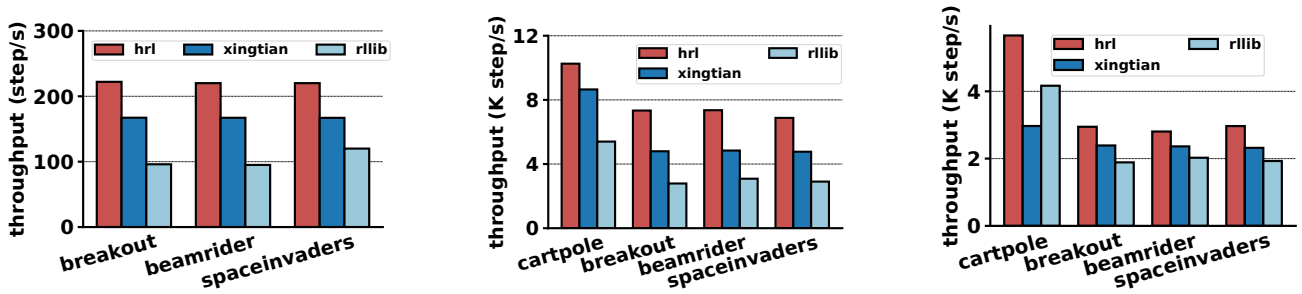
***HRL* brings $18.7\%-90.6\%$ increase in PPO's throughput by optimizing the training procedure and communication channel.** In the PPO scene, the sampling speed is a crucial factor that influences the performance of throughput. Optimizing the training process and communication will benefit the iteration speed of the RL system. And as a result, we can get as high as a 90.6% increase in Gym classic control environment CartPole and 18.7% on average in Atari environments from the optimizations.

***HRL* brings $18.6\% - 52.9\%$ increase in IMPALA's throughput.** Unlike PPO, IMPALA need not wait for the termination of all sampling nodes and can get high throughput from this asynchronous training feature. The IMPALA algorithm can benefit from the improvement of learners, increased sampling speed from the pipeline sampler, and communication optimization. As a result, we can achieve a 52.9% increase in the throughput of Gym environment CartPole and an 18.6% increase in Atari environments.

***HRL* outperforms the local optimization system of Sample Factory in the overall throughput.** We also compare the throughput of *HRL* with the *explorer*-local optimization system of Sample Factory. As shown in Fig. 7a, *HRL* outperforms Sample Factory in APPO algorithm, achieving 30%-40% higher throughput.

C. Convergence rate

***HRL* has a faster convergent speed than *XingTian*, with a speed increase of 36% to 41% in different algorithm scenarios.** Fig. 6 shows the convergent performance of *HRL*



(a) DQN (b) IMPALA (c) PPO

Fig. 5. The overall throughput comparison of DQN, IMPALA, PPO algorithms in different simulation environments.

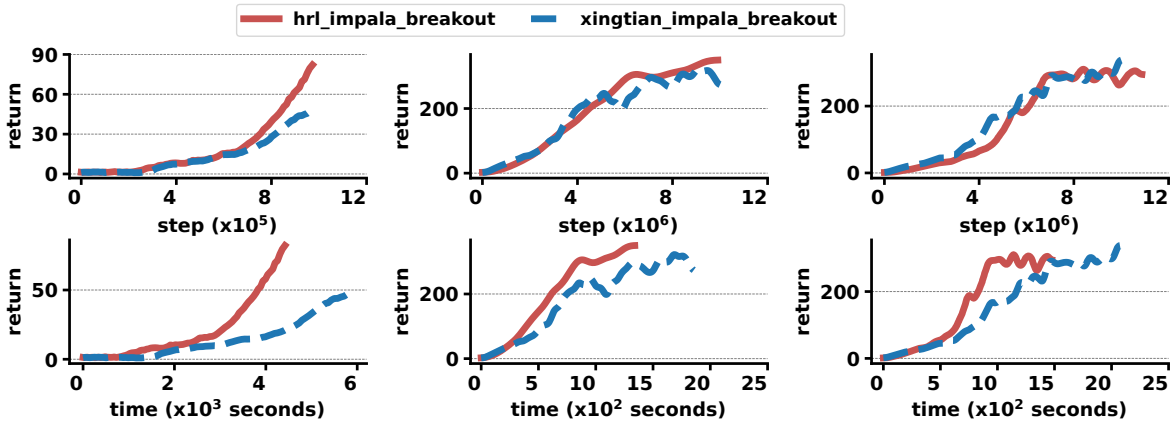
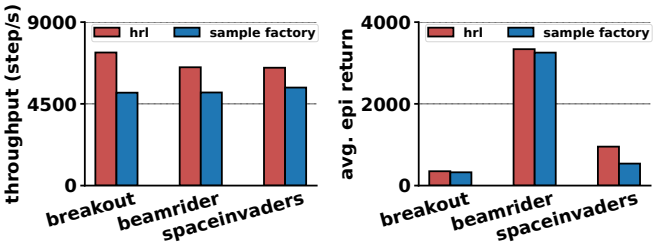


Fig. 6. The convergent speed performance of *HRL* and *XingTian*. The above three figures illustrate the average episode return over the number of training steps. As the training time of each step may change, we further show the average episode return over time in the three figures below.



(a) Throughput (b) Avg. episode return

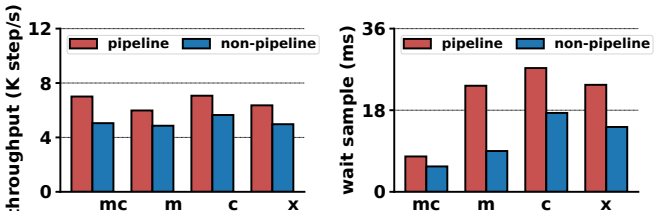
Fig. 7. The performance of *HRL* and Sample Factory

and *XingTian*. As both *HRL* and *XingTian* have much higher throughput than RLlib, we do not consider the RLlib in the experiments. We can see that in terms of steps, *HRL* has a convergent behaviour similar to *XingTian*, but in terms of time, *HRL* has better convergence speed performance than *XingTian*. Achieving the same reward, the time has been reduced by 36% in DQN, 38.5% in PPO and 41% in IMPALA.

Compared with Sample Factory, *HRL* improves the average episode return by 8.0%-77.1%. Fig. 7b shows the final average episode return of RL, and Sample Factory after 10M steps. We can see that in the APPO algorithm, *HRL* has a similar or better reward in different environments. This means our implementation of this APPO algorithm stays the same algorithm architecture and logic. The high throughput can bring better performance to this algorithm. *HRL* gets 8.0%-77.1% more rewards than Sample Factory in different environments.

D. Component Analysis

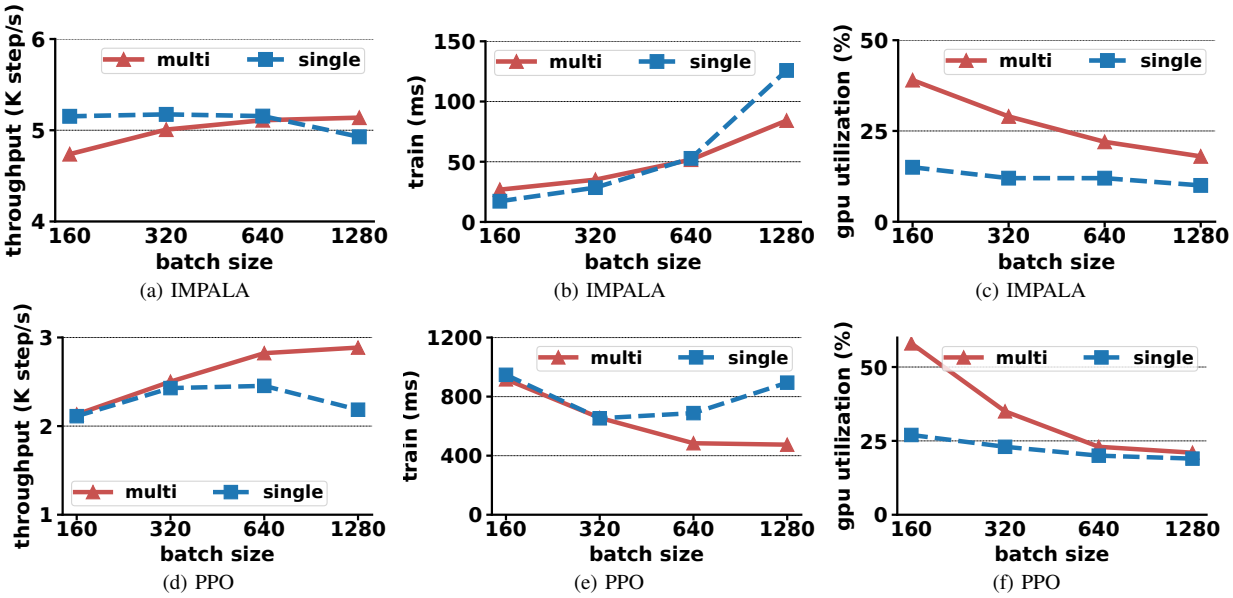
In this section, we evaluate the contribution of each separate portion technique in *HRL*.



(a) Tpt. under different combinations (b) Sample waiting time by learner

Fig. 8. The performance evaluation of pipeline sampler. m refers to the effect of multi-learner, and c refers to communication optimization, x represents the original *XingTian* and mc refers to the combination of muti-learner and communication optimization.

1) Pipeline Sampler: The pipeline sampler module can achieve a higher sample generation speed than the original sampling method. As shown in Fig. 8a, under different combinations of optimization techniques, the pipeline sampler can bring about a 12.2% to 21.5% increase in throughput. Fig. 8b shows the overhead. The pipeline sampling process has added startup expenses, causing an increase in the total sampling time. However, it generates more data and enhances the final data production speed beyond the original *XingTian*.



(a) IMPALA (b) IMPALA (c) IMPALA

(d) PPO (e) PPO (f) PPO

Fig. 9. The performance evaluation of multi-learner module.

2) Multi-Learner: This process introduces additional communication overhead. However, the benefits of the ML module will exceed the overhead when the data volume is large. In the experimental setup, the amount of data used during each training session in the RL task is changed. We can see that the GPU utilization has been improved in both the PPO and Impala algorithm scenarios from Fig. 9. The IMPALA algorithm only uses data from a few sampling nodes during each training, while the PPO uses data from all nodes. Thus, the multi-learner module obtains better benefits in the PPO scenario. We can see that the multi-learner module contributes to a 20.6%-41.3% reduction in training time and an 11.4%-21.2% contribution to overall throughput when the batch size is above 640.

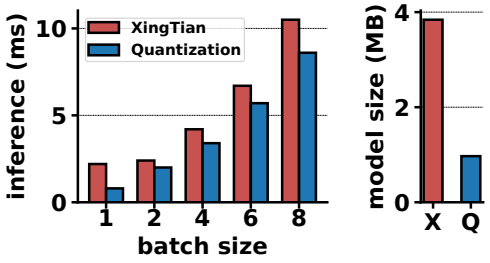


Fig. 10. X refers to the original model size and Q refers to the model size after quantization. Batch size here refers to environments in a group

3) Quantization: Quantization optimizes model size and reduces pressure on network bandwidth caused by model weight synchronization. Fig. 10 shows the effect of quantization on model size compression, compressing the model size of IMPALA from 4MB to around 1 MB. In addition, quantized smaller models exhibit better computational performance during inference tasks. In Fig. 10, we see a 15.2%-50.4% reduction in inference time for different inference batch sizes.

VI. Related Work

Various system-level abstractions have been developed to facilitate the development of RL applications, including Acme [10], RLzoo [30], CleanRL [31], and ElegantRL [15]. While these works have been effective in reducing development costs and improving scalability, they do not fully utilize available resources. Hence, they can enhance their throughput using *HRL*.

Several works have focused on optimizing the speed of reinforcement learning sampling. For example, Sample Factory [14] improves the sampling throughput through parallelism and GPU-based inference in multiple control problems. Similarly, ActorQ [12] has introduced a novel training paradigm to accelerate actor-learner distributed training by utilizing quantized actors. EnvPool [27], has optimized the speed of environment simulations on various hardware setups. Despite their significant contributions, these works have not considered the end-to-end impact, including communication and training. *HRL* can address this gap by optimizing the speed of the entire system.

Several optimization techniques have been applied to reinforcement learning algorithms to improve their efficiency. For example, GA3C [13] relocates the learning calculation component to the GPU and the data collection environment interaction component to the CPU. This architectural change makes the system more compact and efficient, matching the hardware architecture of other deep learning tasks. IMPALA [8] introduces V-trace, which corrects value errors and enables offline learning, allowing for greater throughput and increased policy lag. In contrast to synchronous algorithms like PPO [20], APPO [14] uses asynchronous sampling, making it more efficient in sampling. These algorithm-level optimizations improve sampling efficiency by reusing sampled data. However, our proposed design focuses on improving scalability and

increasing sampling speed, which is applicable to multiple algorithms.

VII. Conclusion

HRL addresses the throughput limitations of traditional RL systems by optimizing the three major aspects of the training process: *learner*, *explorer* and *network*. *HRL* implements a comprehensive optimization system that can improve the overall system throughput by up to 90.6%. The improvements come from efficient sampling methods and addressing bottlenecks in the network communication and learning agent. The results of the experiments show that *HRL* can significantly improve the performance of the DQN, PPO, and IMPALA algorithms. This system has been integrated with *XingTian* and has been proven to be effective in increasing the speed of RL model convergence.

VIII. Acknowledgement

This work is supported in part by the National Key Research and Development Program of China No. 2022YFB4500702; the Open Research Projects of Zhejiang Lab (NO. 2021DA0AM01/003); Shandong Provincial Natural Science Foundation (NO. ZR2022LZH018); National Natural Science Foundation of China under grant 62141218.

References

[1] D. Silver, A. Huang, C. J. Maddison, A. Guez *et al.*, "Mastering the game of go with deep neural networks and tree search," *nature*, vol. 529, no. 7587, pp. 484–489, 2016.

[2] G. Weisz, P. Budzianowski, P.-H. Su, and M. Gai, "Sample efficient deep reinforcement learning for dialogue systems with large action spaces," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 26, no. 11, pp. 2083–2097, 2018.

[3] X. Hou, J. Zhang, C. He, Y. Ji, J. Zhang, and J. Han, "Autonomous driving at the handling limit using residual reinforcement learning," *Advanced Engineering Informatics*, vol. 54, p. 101754, 2022.

[4] S. Gu, E. Holly, T. Lillicrap, and S. Levine, "Deep reinforcement learning for robotic manipulation with asynchronous off-policy updates," in *2017 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2017, pp. 3389–3396.

[5] O. Vinyals, I. Babuschkin, W. M. Czarnecki, M. Mathieu, A. Dudzik, J. Chung, D. H. Choi *et al.*, "Grandmaster level in starcraft ii using multi-agent reinforcement learning," *Nature*, vol. 575, no. 7782, pp. 350–354, Nov 2019.

[6] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous methods for deep reinforcement learning," in *International conference on machine learning*. PMLR, 2016, pp. 1928–1937.

[7] B. Osiski, A. Jakubowski, P. Zicina, P. Mio, C. Galias, S. Homoceanu, and H. Michalewski, "Simulation-based reinforcement learning for real-world autonomous driving," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, 2020, pp. 6411–6418.

[8] L. Espeholt, H. Soyer, R. Munos, K. Simonyan, V. Mnih, T. Ward, Y. Doron, V. Firoiu, T. Harley, I. Dunning *et al.*, "Impala: Scalable distributed deep-rl with importance weighted actor-learner architectures," in *International conference on machine learning*. PMLR, 2018, pp. 1407–1416.

[9] E. Liang, R. Liaw, R. Nishihara, P. Moritz, R. Fox, K. Goldberg, J. Gonzalez, M. Jordan, and I. Stoica, "Rllib: Abstractions for distributed reinforcement learning," in *International Conference on Machine Learning*. PMLR, 2018, pp. 3053–3062.

[10] M. Hoffman, B. Shahriari, J. Aslanides, G. Barth-Maron, F. Behbahani, T. Norman, A. Abdolmaleki, A. Cassirer, F. Yang, K. Baumli *et al.*, "Acme: A research framework for distributed reinforcement learning," *arXiv preprint arXiv:2006.00979*, 2020.

[11] L. Espeholt, R. Marinier, P. Stanczyk, K. Wang, and M. Michalski, "Seed rl: Scalable and efficient deep-rl with accelerated central inference," *arXiv preprint arXiv:1910.06591*, 2019.

[12] M. Lam, S. Chitlangia, S. Krishnan, Z. Wan, G. Barth-Maron, A. Faust, and V. J. Reddi, "Actorq: Quantization for actor-learner distributed reinforcement learning," in *ICLR*, Hardware Aware Efficient Training Workshop at ICLR 2021, Virtual, May 7, 2021.

[13] M. Babaeizadeh, I. Frosio, S. Tyree, J. Clemons, and J. Kautz, "Ga3c: Gpu-based a3c for deep reinforcement learning," *CoRR abs/1611.06256*, 2016.

[14] A. Petrenko, Z. Huang, T. Kumar, G. Sukhatme, and V. Koltun, "Sample factory: Egocentric 3d control from pixels at 100000 fps with asynchronous reinforcement learning," in *International Conference on Machine Learning*. PMLR, 2020, pp. 7652–7662.

[15] X.-Y. Liu, Z. Li, Z. Yang, J. Zheng, Z. Wang, A. Walid, J. Guo, and M. I. Jordan, "Elegantrl-podracer: Scalable and elastic library for cloud-native deep reinforcement learning," *arXiv preprint arXiv:2112.05923*, 2021.

[16] R. Han, J. Demmel, and Y. You, "Auto-precision scaling for distributed deep learning," in *High Performance Computing*, B. L. Chamberlain, A.-L. Varbanescu, H. Ltaief, and P. Luszczek, Eds. Cham: Springer International Publishing, 2021, pp. 79–97.

[17] J. Björck, X. Chen, C. De Sa, C. P. Gomes, and K. Weinberger, "Low-precision reinforcement learning: Running soft actor-critic in half precision," in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, M. Meila and T. Zhang, Eds., vol. 139. PMLR, 18–24 Jul 2021, pp. 980–991.

[18] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, "Openai gym," 2016.

[19] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, "Playing atari with deep reinforcement learning," *arXiv preprint arXiv:1312.5602*, 2013.

[20] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[21] L. Pan, J. Qian, W. Xia, H. Mao, J. Yao, P. Li, and Z. Xiao, "Optimizing communication in deep reinforcement learning with xingtian," in *Proceedings of the 23rd Conference on 23rd ACM/IFIP International Middleware Conference*, ser. Middleware '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 255268.

[22] "TensorFlow: Large-scale machine learning on heterogeneous systems," https://www.tensorflow.org/, Google, 2015.

[23] "Mindspore," https://www.mindspore.cn/, Huawei, 2021.

[24] R. S. Sutton and A. G. Barto, "Reinforcement learning: An introduction," *IEEE Transactions on Neural Networks*, vol. 9, no. 5, p. 1054, 1998.

[25] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski *et al.*, "Human-level control through deep reinforcement learning," *nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[26] Y. Jiang, Y. Zhu, C. Lan, B. Yi, Y. Cui, and C. Guo, "A unified architecture for accelerating distributed dnn training in heterogeneous gpu/cpu clusters," in *Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation*, 2020, pp. 463–479.

[27] J. Weng, M. Lin, S. Huang, B. Liu, D. Makoviichuk, V. Makoviychuk, Z. Liu, Y. Song, T. Luo, Y. Jiang, Z. Xu, and S. Yan, "EnvPool: A highly parallel reinforcement learning environment execution engine," *arXiv preprint arXiv:2206.10558*, 2022.

[28] L. C. Dongdong Chen, Junshi Huang and T. Ma, "Kungfu: A high-performance distributed training framework for deep learning," 2020.

[29] P. Moritz, R. Nishihara, S. Wang, A. Tumanov, R. Liaw, E. Liang, M. Elibol, Z. Yang, W. Paul, M. I. Jordan *et al.*, "Ray: A distributed framework for emerging {AI} applications," in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 2018, pp. 561–577.

[30] Z. Ding, T. Yu, H. Zhang, Y. Huang, G. Li, Q. Guo, L. Mai, and H. Dong, "Efficient reinforcement learning development with rlzoo," in *Proceedings of the 29th ACM International Conference on Multimedia*, 2021, pp. 3759–3762.

[31] S. Huang, R. F. J. Dossa, C. Ye, and J. Braga, "Cleanrl: High-quality single-file implementations of deep reinforcement learning algorithms," *arXiv preprint arXiv:2111.08819*, 2021.